ISLAMIC UNIVERSITY OF TECHNOLOGY

SWE 4301

LECTURE 5

Topic: SOLID & Open-Closed Principle

September 26, 2024

PREPARED BY

Maliha Noushin Raida

Lecturer, Department of CSE

Contents

1 SOLID Design Principles in Software Development **3**

2 OCP: OpenClosed Princple **3**

2.1 Abstraction is the key 4

2.2 Violating OCP Example 5

2.3 Design maintaining OCP 6

1 SOLID Design Principles in Software Development

SOLID is a set of five design principles. These principles help software developers design robust, testable, extensible, and maintainable object-oriented software systems. Each of these five design principles solves a particular problem that might arise while developing the software systems. Discuss why principles are important

1. Principles keep everyone in order
2. Principles are well established and proven
3. Following the principles gives consistency

Mention the SOLID principles. The five fundamental principles of OOP.

- S - Single Responsibility Principle (SRP)
- O - Open/Closed Principle (OCP)
- L - Liskov Substitution Principle (LSP)
- I - Interface Segregation Principle (ISP)
- D - Dependency Inversion Principle (DIP)

2 OCP: OpenClosed Princple

All systems change during their life cycles. This must be born in mind when developing systems are expected to last longer than the first version

How can we create designs that are stable in the face of change and that will last longer than the first version? Bertrand Meyer gave us guidance as long ago as 1988 when he coined the now famous open-closed principle. To paraphrase him:

Software entities (classes, modules, functions, etc.) should be open for extension, but closed for modification.

When a single change to a program results in a cascade of changes to dependent modules, that program exhibits the undesirable attributes that we have come to associate with “bad” design. The program becomes fragile, rigid, unpredictable and un-reusable. The open/closed principle attacks this in a very straightforward way. It says that you should design modules that never change . When requirements change, you extend the behavior of such modules by adding new code, not by changing old code that already works.

- They are "Open For Extension". This means that the behavior of the module can be extended. That we can make the module behave in new and different ways as the requirements of the application change, or to meet the needs of new applications.
- They are "Closed for Modification". The source code of such a module is inviolate. No one is allowed to make source code changes to it.

It would seem that these two attributes are at odds with each other. The normal way to extend the behavior of a module is to make changes to that module. A module that cannot be changed is normally thought to have a fixed behavior.

Conformance to this principle is what yields the greatest benefits claimed for object oriented technology; i.e. re-usability and maintainability. Yet conformance to this principle is not achieved simply by using an object oriented programming language. Rather, it requires a dedication on the part of the designer to apply abstraction to those parts of the program that the designer feels are going to be subject to change.

By adhering to OCP, this encourages loose coupling between components that are closed modifications and new functionality that is added. Usually, this is accomplished when components depend on interface instead of concrete classes. An interface acts as a layer of abstraction where well-defined components do not have to worry about the underlying changes made by the concrete classes that implement the interface. Therefore, this makes the program easy to maintain.

2.1 Abstraction is the key

It is possible to create abstractions that are fixed and yet represent an unbounded group of possible behaviors. The abstractions are **abstract base classes**, and the unbounded group of possible behaviors is **represented by all the possible derivative classes**. It is possible for a module to manipulate an abstraction. Such a module can be closed for modification since it depends upon an abstraction that is fixed. Yet the behavior of that module can be extended by creating new derivatives of the abstraction. Both the `Client` and `Server` classes

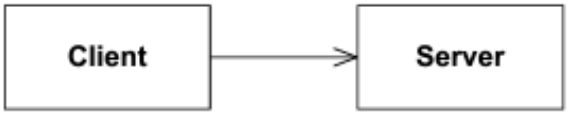

Figure 1: `Client` and `Server` Class without Abstraction

are concrete. The `Client` class uses the `Server` class. If we wish for a `Client` object to use

a different `Server` object, then the `Client` class must be changed to name the new `Server` class.

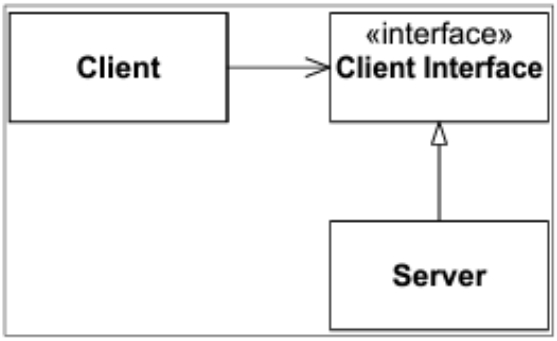

Figure 2: `Client` and `Server` Class with Abstraction

In this case, the `ClientInterface` class is an abstract class with abstract member functions. The `Client` class uses this abstraction; however, objects of the `Client` class will be using objects of the derivative Server class. If we want `Client` objects to use a different server class, then a new derivative of the `ClientInterface` class can be created. The `Client` class can remain unchanged.

2.2 Violating OCP Example

```
enum EmployeeType {
    Permanent, Contructual, PartTime
}

class Employee {
    private readonly EmployeeType type;
    private int transportAllowance;

    public Employee(EmployeeType type, int transportAllowance) {
        this.type = type;
        this.transportAllowance = transportAllowance;
    }

    public int CalculateSalary(int basic, int phr, int hw) {
        if (type == EmployeeType.Permanent) {
            return basic + basic / 4 + transportAllowance;
        } else if (type == EmployeeType.Contructual) {
            return basic + transportAllowance + phr * hw;
```

```
        } else {
            return basic + phr * hw;
        }
    }

    public int CalculateBonus(int basic) {
        if (type == EmployeeType.Permanent) {
            return basic / 2;
        } else if (type == EmployeeType.PartTime) {
            return 0;
        } else {
            return 5000;
        }
    }
}
```

Listing 1: A `Employee` class in Java

How do you handle changing requirements and feedback that may affect the open-closed principle in OOP?

- **Use abstraction and inheritance**: One way to adhere to the open-closed principle is to use abstraction and inheritance to create a hierarchy of classes that share common features but can also have specific behaviors. Abstraction is the process of hiding the details of how something works and exposing only the essential interface. Inheritance is the mechanism that allows a subclass to inherit the attributes and methods of a superclass and override or extend them as needed.

- **Use interfaces and polymorphism**: Another way to follow the open-closed principle is to use interfaces and polymorphism to define contracts and behaviors for different types of objects. An interface is a collection of abstract methods that specify what an object can do, but not how it does it. Polymorphism is the ability of an object to take different forms and behave differently depending on the context. For example, you can create an interface called Flyable that defines a method called fly, and then implement this interface in different classes like Bird, Airplane, and Superman. This way, you can use polymorphism to call the fly method on any object that implements Flyable, without knowing or changing its internal details.

2.3 Design maintaining OCP

```
enum EmployeeType {
```

```
    PERMANENT, CONTRACTUAL, PARTTIME
}

abstract class Employee {
    protected int transportAllowance;

    public Employee(int transportAllowance) {
        this.transportAllowance = transportAllowance;
    }

    public abstract int calculateSalary(int basic, int phr, int hw);

    public abstract int calculateBonus(int basic);
}

class PermanentEmployee extends Employee {

    public PermanentEmployee(int transportAllowance) {
        super(transportAllowance);
    }

    @Override
    public int calculateSalary(int basic, int phr, int hw) {
        return basic + basic / 4 + transportAllowance;
    }

    @Override
    public int calculateBonus(int basic) {
        return basic / 2;
    }
}


class ContractualEmployee extends Employee {
    public ContractualEmployee(int transportAllowance) {
        super(transportAllowance);
    }
    @Override
    public int calculateSalary(int basic, int phr, int hw) {
        return basic + transportAllowance + phr * hw;
    }
    @Override
```

```java
    public int calculateBonus(int basic) {
        return 5000;
    }
}
class PartTimeEmployee extends Employee {
    public PartTimeEmployee(int transportAllowance) {
        super(transportAllowance);
    }
    @Override
    public int calculateSalary(int basic, int phr, int hw) {
        return basic + phr * hw;
    }
    @Override
    public int calculateBonus(int basic) {
        return 0;
    }
}

public class Main {
    public static void main(String[] args) {
        Employee permanentEmployee = new PermanentEmployee(3000);
        Employee contractualEmployee = new ContractualEmployee(2000);
        Employee partTimeEmployee = new PartTimeEmployee(1000);

        // Example calculations
        System.out.println("Permanent Employee Salary: " +
    permanentEmployee.calculateSalary(50000, 200, 160));
        System.out.println("Permanent Employee Bonus: " +
    permanentEmployee.calculateBonus(50000));

        System.out.println("Contractual Employee Salary: " +
    contractualEmployee.calculateSalary(30000, 150, 100));
        System.out.println("Contractual Employee Bonus: " +
    contractualEmployee.calculateBonus(30000));

        System.out.println("Part-Time Employee Salary: " +
    partTimeEmployee.calculateSalary(20000, 100, 80));
        System.out.println("Part-Time Employee Bonus: " +
    partTimeEmployee.calculateBonus(20000));
    }
}
```

Listing 2: Design reconstructed in Java

We solved the violation using abstract class and polymorphism.

In general, no matter how “closed” a module is, there will always be some kind of change against which it is not closed. Since closure cannot be complete, it must be strategic. That is, the designer must choose the kinds of changes against which to close his design. This takes a certain amount of prescience derived from experience. The experienced designer knows the users and the industry well enough to judge the probability of different kinds of changes. He then makes sure that the open-closed principle is invoked for the most probable changes.